

ASIGNATURA: INGENIERÍA DE SOFTWARE 2
PERÍODO ACADÉMICO: 2025-2
FECHA: 12/11/25
TIEMPO: 100 minutos

NOTA

EXAMEN ESCRITO N°2

CÓDIGO	APELLIDOS Y NOMBRES	SECCIÓN
		851

ANTES DE INICIAR LA EVALUACIÓN DEBE LEER LAS INSTRUCCIONES

Si la evaluación indica cargar algún archivo en la computadora, recuerde que es responsabilidad del estudiante hacerlo en el tiempo establecido y con las instrucciones dadas. Debe indicar el número de la misma en el recuadro siguiente:

INSTRUCCIONES GENERALES:

- La evaluación consta de “5” preguntas, cuyo puntaje está indicado en cada una de ellas.
- El procedimiento, el orden, la claridad de las respuestas y el uso apropiado del lenguaje (notaciones, símbolos y unidades), serán considerados como criterios de calificación.
- Escriba únicamente con lapicero de tinta azul o negra. La evaluación desarrollada con lápiz no será calificada.
- Devolver todo el material entregado.
- **Leer detenidamente las situaciones que ocasionarán la anulación de la evaluación, que se encuentran a continuación.**

SITUACIONES QUE OCASIONARÁN LA ANULACIÓN DE LA EVALUACIÓN:

- Mantener encendidos, visibles o al alcance de la mano dispositivos como teléfonos celulares, relojes smart u otros medios electrónicos de comunicación.
- Utilizar material de consulta no autorizado (apuntes de clase, fotocopias o materiales similares).
- Utilizar calculadora no estando permitido.
- Compartir o intercambiar hojas, tablas, cualquier material impreso, dispositivo electrónico, durante el desarrollo de la evaluación.
- Conversar durante el desarrollo de la evaluación.

Los profesores de la asignatura

FIRMA DEL ALUMNO (LEYÓ LAS INSTRUCCIONES)
--

Nota:

- Para las preguntas que requieran código, debe incluir el **código fuente** directamente en el examen en **formato de texto**, acompañado de **comentarios** que expliquen las acciones realizadas para **validar su respuesta**. Además, debe **adjuntar una captura de pantalla** que muestre la **salida de la ejecución** del programa.
- Finalmente, entregue un archivo comprimido con el nombre **apellidos_EE2.zip**, que contenga ordenado **todo el código correspondiente a sus respuestas**.

Pregunta 1. (6 puntos)

Ingrese al Blackboard para responder las preguntas Teóricas

Pregunta 2. (4 puntos)

Implemente en Java un método que determine si un año dado es **bisiesto** o no. Un año se considera bisiesto si cumple las siguientes condiciones:

- Es divisible entre 4.
- Excepto si es divisible entre 100, en cuyo caso **no** es bisiesto.
- Sin embargo, si es divisible entre 400, **sí** es bisiesto.

Una vez implementado el método, escriba las **pruebas unitarias** necesarias utilizando **JUnit** para comprobar su correcto funcionamiento.

Incluya los siguientes casos de prueba:

- a) Año divisible por 400 (debe ser bisiesto).
- b) Año divisible por 100 pero no por 400 (no debe ser bisiesto).
- c) Año divisible por 4 pero no por 100 (debe ser bisiesto).
- d) Año no divisible por 4 (no debe ser bisiesto).
- e) Un caso límite (por ejemplo, el año 0 o un año negativo).

Pregunta 3. (4 puntos)

Dado el siguiente código fuente en Java, realice la prueba de caja blanca aplicando la técnica de complejidad ciclomatica propuesta por McCabe.

```
public boolean CajaBlanca(int num) {  
    String s = String.valueOf(num);  
    int n = s.length();  
  
    if (n % 2 != 0) return false;  
  
    int mitad = n / 2;  
    int min = (int) Math.pow(10, mitad - 1);  
    int max = (int) Math.pow(10, mitad) - 1;  
  
    for (int i = min; i <= max; i++) {  
        if (num % i != 0)  
            continue;  
        int j = num / i;  
        if (j < min || j > max)  
            continue;  
        if (i % 10 == 0 && j % 10 == 0)  
            continue;  
        String combinado = String.valueOf(i) + String.valueOf(j);  
        if (combinado.length() != n)  
            continue;  
  
        char[] orig = s.toCharArray();  
        char[] comb = combinado.toCharArray();  
        Arrays.sort(orig);  
        Arrays.sort(comb);  
        if (Arrays.equals(orig, comb))  
            return true;  
    }  
    return false;  
}
```

- a) Etiquete el código y muestre el grafo asociado y calcule la complejidad ciclomatica (2 puntos)
- b) Solo se evalúa si la complejidad es correcta: (2 puntos)
 - Determine los caminos independientes
 - Generar los casos de prueba para cada camino independiente

Pregunta 4. (4 puntos)

Se requiere diseñar y aplicar casos de prueba utilizando la técnica de partición de equivalencia para el método `validarFormulario()` que evalúa los datos ingresados por un usuario en un sistema de registro.

El método recibe cinco parámetros y debe devolver `true` si todas las entradas son válidas, o `false` en caso contrario.

Descripción de los campos de entrada:

- **Edad:** Representa la edad del usuario en años. Solo se aceptan edades entre **18 y 65 años inclusive**.
- **Correo electrónico:** Corresponde a la dirección de correo del usuario. Debe tener el formato correcto: incluir nombre de usuario, el símbolo "@" y un dominio (por ejemplo, `usuario@dominio.com`).
- **Tipo de usuario:** Indica el rol o tipo de acceso que tendrá el usuario en el sistema. Solo se aceptan los valores: "ADMIN", "CLIENTE" o "INVITADO".
- **Monto de compra:** Es el monto total de dinero asociado al registro o compra inicial. El monto debe ser **mayor o igual a 0.00** y **menor o igual a 10,000.00**.
- **Acepta términos y condiciones:** Indica si el usuario ha aceptado las políticas del sistema. Solo se considera válido si el valor ingresado es **true** (es decir, si el usuario acepta los términos). Si el valor es **false**, no se permite completar el registro.

```
public static boolean validarFormulario(int edad, String correo, String tipoUsuario, double monto, boolean aceptaTerminos)
```

Se solicita:

- Identificar la clase de equivalencia a la que pertenece cada dato. (1 puntos)
- Crear una Tabla de Clases de Equivalencia (1 puntos)
- Cree la tabla de casos de prueba válidas (1 puntos)
- Cree la tabla de casos de prueba No válidas (1 puntos)

Pregunta 5. (2 puntos)

Una empresa de mensajería desea mejorar su sistema de cálculo de costos de envío. Actualmente, cuenta con una clase llamada Envío que determina el costo total de un paquete según su **tipo** (normal, express o fragil) y su **peso**.

El sistema funciona correctamente, pero el código presenta algunos **problemas de mantenibilidad y acoplamiento**, lo cual dificulta la extensión del sistema cuando se desea agregar nuevos tipos de envío o reglas de negocio.

A continuación, se presenta el siguiente fragmento de código en **Java**:

Envio.java

```
public class Envio {
    String tipoPaquete; // "normal", "express", "fragil"
    double peso;

    public Envio(String tipoPaquete, double peso) {
        this.tipoPaquete = tipoPaquete;
        this.peso = peso;
    }

    public double calcularCosto() {
        double costoBase = peso * 2.0;
        double recargo = 0.0;

        if ("express".equals(tipoPaquete)) {
            recargo = 10.0;
        } else if ("fragil".equals(tipoPaquete)) {
            recargo = 5.0;
        }

        // Validación adicional: si es fragil y pesa más de 10kg, no se envía
        if ("fragil".equals(tipoPaquete) && peso > 10) {
            throw new IllegalArgumentException("Paquetes frágiles no pueden pesar más de 10 kg");
        }

        return costoBase + recargo;
    }
}
```

Main.java

```
import java.util.Arrays;
import java.util.List;

public class Main {
    public static void main(String[] args) {
        // Crear una lista de envíos de prueba
        List<Envio> envios = Arrays.asList(
            new Envio("normal", 5.0),
            new Envio("express", 3.5),
            new Envio("rágil", 8.0),
            new Envio("rágil", 12.0) // Este debe lanzar excepción
        );

        // Recorrer la lista y calcular el costo de cada envío
        for (Envio envio : envios) {
            try {
                double costo = envio.calcularCosto();
                System.out.println("Tipo: " + envio.tipoPaquete +
                    ", Peso: " + envio.peso +
                    " kg -> Costo: S/" + costo);
            } catch (IllegalArgumentException e) {
            }
        }
    }
}
```

```

        System.out.println("Error al calcular envío (" +
            envio.tipoPaquete + ", " +
            envio.peso + " kg): " + e.getMessage());
    }
}
}
}
}

```

EnvioTest.java

```

package pc02;

import static org.junit.Assert.assertEquals;
import org.junit.Test;

public class EnvioTest {

    @Test
    public void testCalcularCostoNormal() {
        Envio envio = new Envio("normal", 5.0);
        double costo = envio.calcularCosto();
        assertEquals(10.0, costo, 0.001);
    }

    @Test
    public void testCalcularCostoExpress() {
        Envio envio = new Envio("express", 3.0);
        double costo = envio.calcularCosto();
        assertEquals(16.0, costo, 0.001);
    }

    @Test
    public void testCalcularCostoFragil() {
        Envio envio = new Envio("fragil", 2.5);
        double costo = envio.calcularCosto();
        assertEquals(10.0, costo, 0.001);
    }

    @Test(expected = IllegalArgumentException.class)
    public void testFragilConPesoMayorA10DebeLanzarExcepcion() {
        Envio envio = new Envio("fragil", 12.0);
        envio.calcularCosto(); // debe lanzar excepción
    }

    @Test
    public void testTipoDesconocidoNoAplicaRecargo() {
        Envio envio = new Envio("premium", 4.0);
        double costo = envio.calcularCosto();
        assertEquals(8.0, costo, 0.001);
    }

    @Test
    public void testPesoCeroDebeDarCostoCero() {
        Envio envio = new Envio("normal", 0.0);
        double costo = envio.calcularCosto();
        assertEquals(0.0, costo, 0.001);
    }
}

```

Se solicita:

- Identifique los **bad smells** presentes en el código y **aplique las técnicas de refactorización adecuadas** para mejorar su estructura. Asegúrese de que el código resultante **mantenga la funcionalidad original** y **pase correctamente las pruebas unitarias**.